

ECS Project Step 2.2 – Testing Docker Image

To Begin with the Lab

Summary of the Lab

In this lab, the Docker image created in the previous step was run locally on a Docker-enabled **Amazon EC2** instance to verify that the PHP application was working before deployment to **Amazon ECS**. The container was started with runtime environment variables so the **Amazon S3** bucket name and AWS Region could be passed dynamically instead of being hardcoded in the source code. The application was then tested by uploading an image, and the uploaded file was confirmed in the configured **Amazon S3** bucket. The video also explained why AWS Access Keys should not be stored inside application code and introduced **IAM Roles** as the recommended best practice for secure AWS access.

Understanding Environment Variables and IAM Roles

Environment variables let an application receive values like a bucket name or Region when the container starts, instead of storing those values inside the code. This exists so the same Docker image can be reused in different environments without rebuilding it every time a setting changes. **IAM Roles** provide temporary AWS permissions to a running application without exposing long-term Access Keys or Secret Keys in the source code. For example, a PHP app that uploads images to **Amazon S3** can use an environment variable for the bucket name and an **IAM Role** for secure access, which keeps the deployment flexible and safer.

Example:

A PHP application uploads customer profile images to a bucket named cloudfolks-lab-bucket in the ap-south-1 Region. Instead of editing upload.php each time the bucket or Region changes, the values are passed into the container at runtime. Later, when the app runs on **Amazon ECS**, the Task uses an **IAM Role** so the application can upload to **Amazon S3** without hardcoded credentials.

Lab Steps

Step 1 – Verify the Docker Image

- Sign in to the **AWS Management Console**.
- Connect to the Docker-enabled **Amazon EC2** build machine using SSH.

Create bucket [info](#)

Buckets are containers for data stored in S3.

General configuration

AWS Region
US East (N. Virginia) us-east-1

Bucket type [info](#)

☒ **General purpose**
Recommended for most use cases and access patterns. General purpose buckets are the original S3 bucket type. They allow a mix of storage classes that redundantly store objects across multiple Availability Zones.

☐ **Directory**
Recommended for low-latency use cases. These buckets use only the S3 Express One Zone storage class, which provides faster processing of data within a single Availability Zone.

Bucket namespace
Choose the namespace where you want to create your bucket. [Learn more](#) [L](#)

☒ **Global namespace**
By default, S3 creates general purpose buckets in the global namespace.

☐ **Account Regional namespace (recommended)**
General purpose buckets created in your account Regional namespace are unique to your account. These buckets can never be created by another AWS account.

Bucket name [info](#)

cloudfreeks-lab-bucket

Bucket names must be 3 to 63 characters and unique within the global namespace. Bucket names must also begin and end with a letter or number. Valid characters are a-z, 0-9, periods (.), and hyphens (-). [Learn more](#) [L](#)

Copy settings from existing bucket - optional
Only the bucket settings in the following configuration are copied.

[Choose bucket](#)

Format: s3://bucket/prefix

- Pass the **Amazon S3** bucket name as an environment variable.
- Pass the **AWS Region** as an environment variable.
- Generate an **Access Key** if you have not made it yet.

Retrieve access keys [info](#)

Access key
If you lose or forget your secret access key, you cannot retrieve it. Instead, create a new access key and make the old key inactive.

Access key

Secret access key

[Show](#)

Access key best practices

- Never store your access key in plain text, in a code repository, or in code.
- Disable or delete access key when no longer needed.
- Enable least-privilege permissions.
- Rotate access keys regularly.

For more details about managing access keys, see the [best practices for managing AWS access keys](#).

[Download .csv file](#) [Done](#)

- Pass the **AWS Access Key** and **Secret Access Key** temporarily for local testing.

```
ec2-user@ip-172-31-33-19:~$ docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
cloudfreeks-s3-php  latest         b31be780b9c5   25 hours ago   541MB

[ec2-user@ip-172-31-33-19 ~]$ docker run -d -p 8080:80 \
-e S3_BUCKET=cloudfreeks-s3-php \
-e AWS_REGION=us-east-1 \
-e AWS_ACCESS_KEY_ID= \
-e AWS_SECRET_ACCESS_KEY= \
--name php-test cloudfreeks-s3-php
9c9eae0ccf8ba0b6e120354b0b69ca4cfae4ca57276cf0852d20a76deaf5b64
[ec2-user@ip-172-31-33-19 ~]$
```

Step 3 – Verify the Running Container

- Verify that the container is running.

docker ps

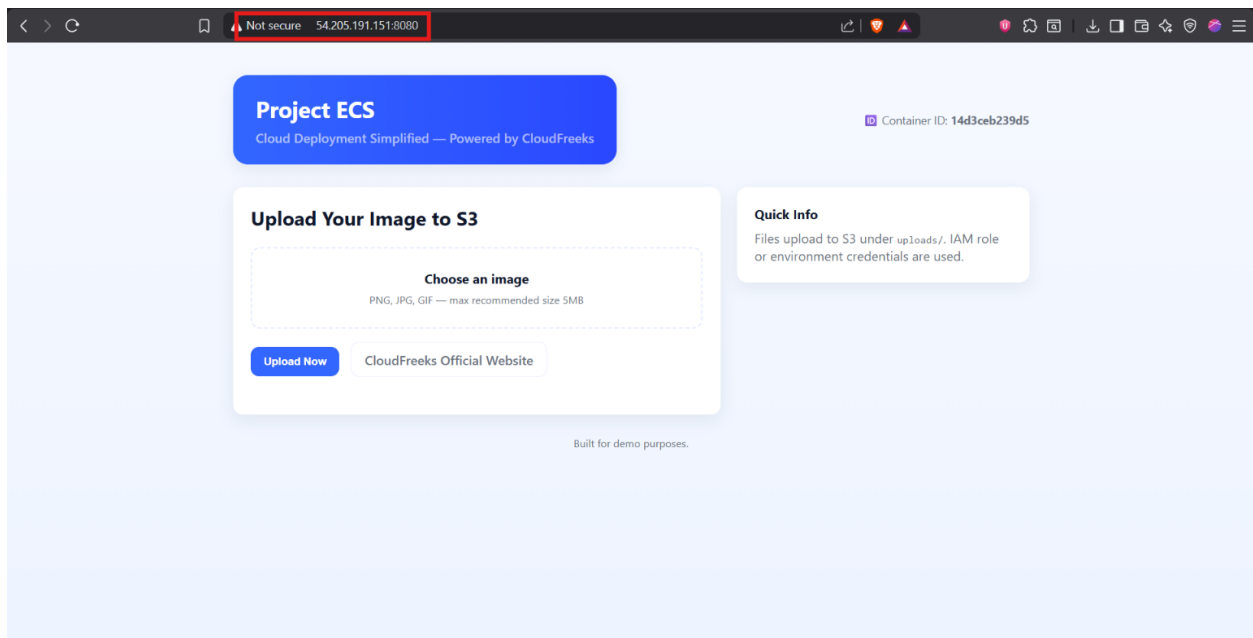
```
ec2-user@ip-172-31-33-19:~$ docker images
REPOSITORY          TAG             IMAGE ID        CREATED         SIZE
cloudfreeks-s3-php  latest         b31be780b9c5   25 hours ago   541MB

[ec2-user@ip-172-31-33-19 ~]$ docker run -d -p 8080:80 \
-e S3_BUCKET=cloudfreeks-s3-php \
-e AWS_REGION=us-east-1 \
-e AWS_ACCESS_KEY_ID= \
-e AWS_SECRET_ACCESS_KEY= \
--name php-test cloudfreeks-s3-php
9c9eae0ccf8ba0b6e120354b0b69ca4cfae4ca57276cf0852d20a76deaf5b64
[ec2-user@ip-172-31-33-19 ~]$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
9c9eae0ccf8   cloudfreeks-s3-php                 "docker-php-entrypoi..." 58 seconds ago Up 58 seconds 8080/tcp, 0.0.0.0:8080->80
/tcp, ::8080->80/tcp   php-test
[ec2-user@ip-172-31-33-19 ~]$
```

- Open the application using the EC2 public IP on port **8080**.

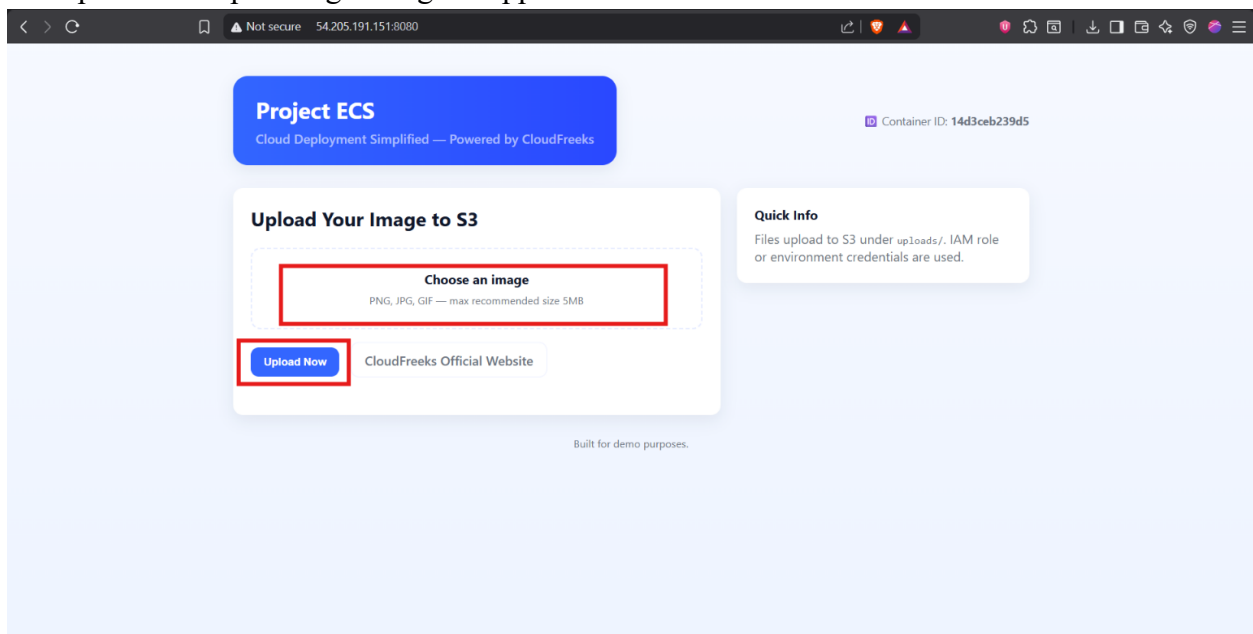
http://<EC2-Public-IP>:8080

- If the application does not open, navigate to **Amazon EC2** → **Security Groups** and confirm that port **8080** is allowed in the inbound rules.

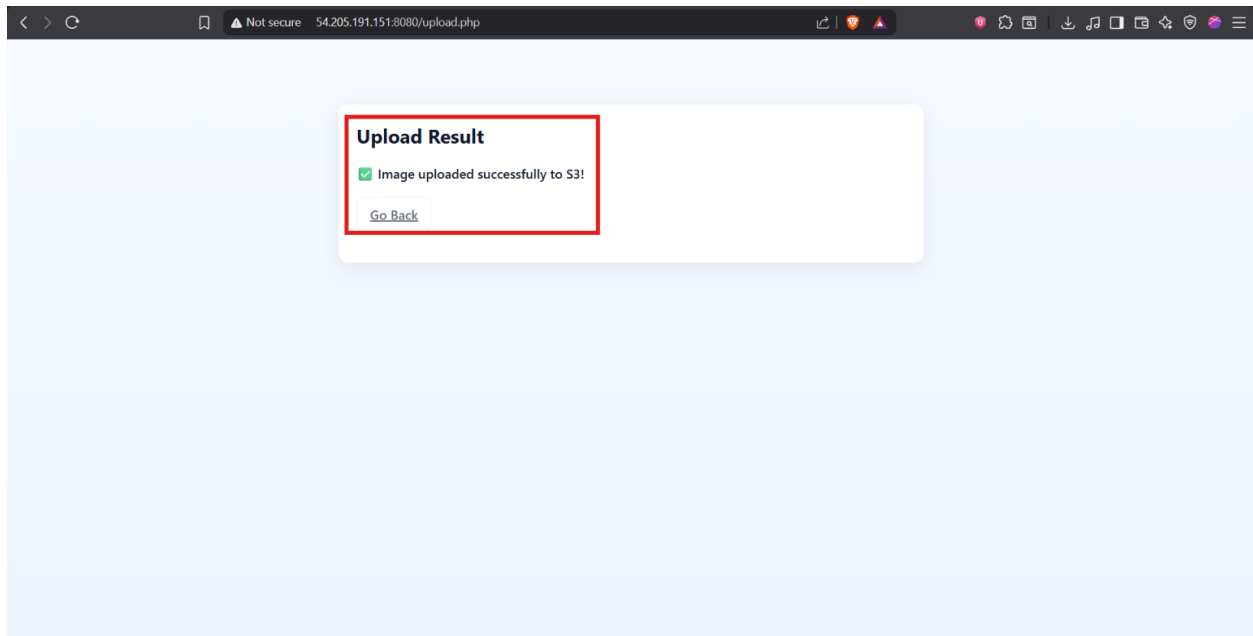


Step 4 – Test the PHP Application

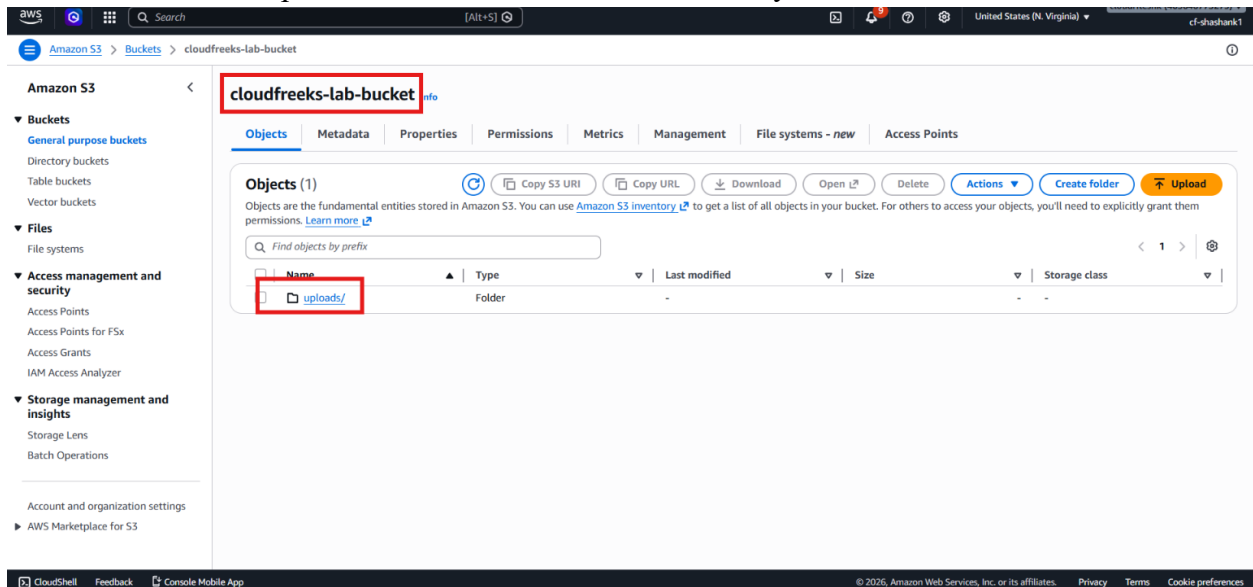
- Upload a sample image using the application.



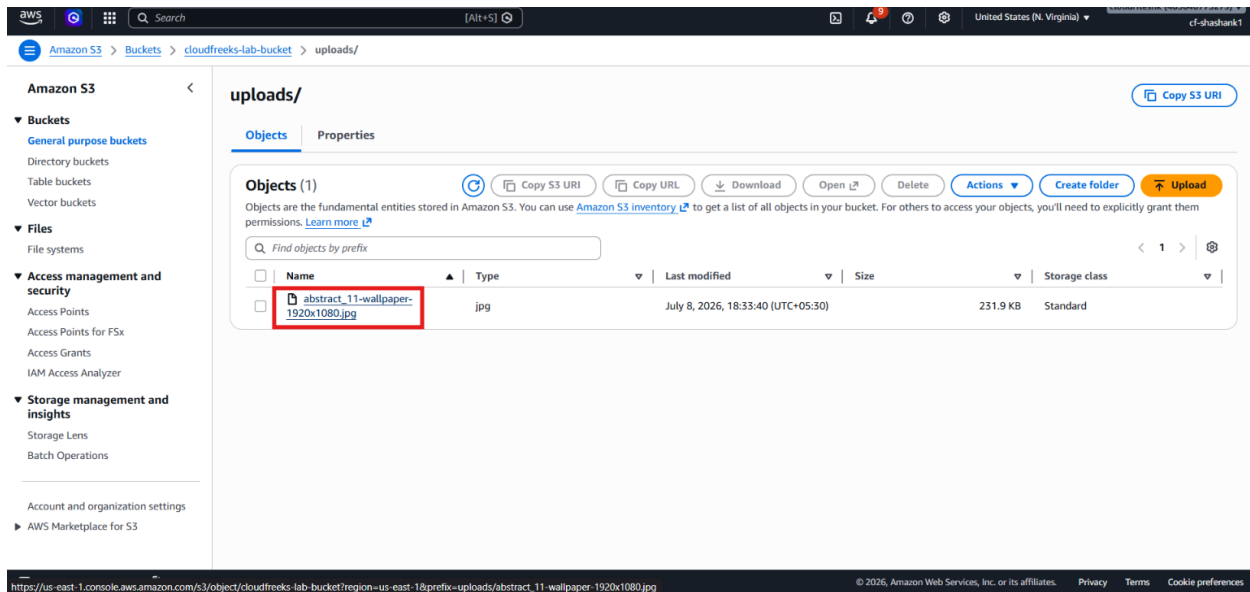
- You get successful upload screen after uploading.



- Navigate to **Amazon S3**.
- Open the configured bucket.
- Confirm that an uploads folder was created automatically.



- Confirm that the uploaded image appears in the bucket and can be opened successfully.



Verification

- Verified that the Docker image was available on the build machine.
- Verified that the Docker container started successfully.
- Verified that the application opened through port **8080**.
- Verified that the image upload succeeded.
- Verified that the uploaded file appeared in the **Amazon S3** bucket.